AD23511-DEEP LEARNING LABORATORY

DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND DATASCIENCE

AD23511 DEEP LEARNING LAB

V SEMESTER – III YEAR

LAB MANUAL ACADEMIC

YEAR 2023-24

SYLLABUS

`AD23511 DEEP LEARNING LABORATORY

LTPC
0042

COURSE OBJECTIVES:

- To understand the tools and techniques to implement deep neural networks
- To apply different deep learning architectures for solving problems
- To implement generative models for suitable applications
- To learn to build and validate different models

LIST OF EXPERIMENTS:

1. Solving XOR problem using DNN
2. Character recognition using CNN
3. Face recognition using CNN
4. Language modeling using RNN
5. Sentiment analysis using LSTM
6. Parts of speech tagging using Sequence to Sequence architecture
7. Machine Translation using Encoder-Decoder model
8. Image augmentation using GANs
9. Mini-project on real world applications

COURSE OUTCOMES:

At the end of this course, the students will be able to:

- CO1: Apply deep neural network for simple problems (K3)
- CO2: Apply Convolution Neural Network for image processing (K3)
- CO3: Apply Recurrent Neural Network and its variants for text analysis (K3)
- CO4: Apply generative models for data augmentation (K3)
- CO5: Develop real-world solutions using suitable deep neural networks (K4)

TABLE OF CONTENTS

S.NO.	TITLE OF THE EXPERIMENTS	PAGE NO.
1	Solving XOR problem using DNN	1
2	Character recognition using CNN	4
3	Face recognition using CNN	10
4	Language modeling using RNN	17
5	Sentiment analysis using LSTM	22
6	Parts of speech tagging using Sequence to Sequence architecture	28
7	Machine Translation using Encoder-Decoder model	37
8	Image augmentation using GANs	43
9	Mini-project on real world applications	50

Exercise No. 1: Solving XOR Problem using Deep Neural Networks

Aim:

To implement a deep neural network (DNN) model to solve the XOR classification problem, which is not linearly separable.

Algorithm:

1. Define the XOR input-output pairs.
2. Initialize weights and biases for input, hidden, and output layers.
3. Use sigmoid activation and forward propagation.
4. Apply backpropagation using gradient descent to minimize loss.
5. Plot the cost over iterations.

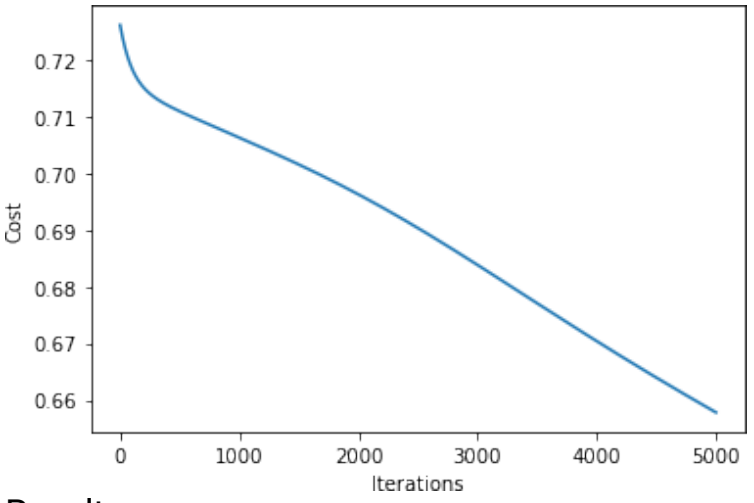
Program:

```
import numpy as np  # For matrix math
import matplotlib.pyplot as plt  # For plotting
import sys  # For printing

# The training data
X = np.array([
    [0, 1],
    [1, 0],
    [1, 1],
    [0, 0]
])

# The labels for the training data
y = np.array([
    [1],
    [1],
    [0],
    [0]
])

# Neural network architecture
num_i_units = 2  # Number of input units
num_h_units = 2  # Number of hidden units
num_o_units = 1  # Number of output units

# Hyperparameters
learning_rate = 0.01
reg_param = 0  # Regularization parameter
max_iter = 5000
m = 4  # Number of training examples

# Seed for reproducibility
np.random.seed(1)
```

```
# Weight and bias initialization
W1 = np.random.normal(0, 1, (num_h_units, num_i_units))  # 2x2
W2 = np.random.normal(0, 1, (num_o_units, num_h_units))  # 1x2
B1 = np.random.random((num_h_units, 1))  # 2x1
B2 = np.random.random((num_o_units, 1))  # 1x1

# Sigmoid function and its derivative
def sigmoid(z, derv=False):
    if derv:
        return z * (1 - z)
    return 1 / (1 + np.exp(-z))

# Forward propagation
def forward(x, predict=False):
    a1 = x.reshape(x.shape[0], 1)  # Column vector
    z2 = W1.dot(a1) + B1  # 2x1
    a2 = sigmoid(z2)  # 2x1
    z3 = W2.dot(a2) + B2  # 1x1
    a3 = sigmoid(z3)  # 1x1
    if predict:
        return a3
    return a1, a2, a3

# Cost tracking
cost = np.zeros((max_iter, 1))

# Training function
def train(_W1, _W2, _B1, _B2):
    for i in range(max_iter):
        c = 0
        dW1 = 0
        dW2 = 0
        dB1 = 0
        dB2 = 0

        for j in range(m):
            sys.stdout.write(f"\rIteration: {i + 1} and {j + 1}")
            a0 = X[j].reshape(X[j].shape[0], 1)  # 2x1

            # Forward pass
            z1 = _W1.dot(a0) + _B1  # 2x1
            a1 = sigmoid(z1)  # 2x1
            z2 = _W2.dot(a1) + _B2  # 1x1
            a2 = sigmoid(z2)  # 1x1

            # Backward pass
            dz2 = a2 - y[j]  # 1x1
            dW2 += dz2 * a1.T  # 1x2
            dz1 = np.multiply((_W2.T * dz2), sigmoid(a1, derv=True))  # 2x1
```

```
            dW1 += dz1.dot(a0.T)  # 2x2
            dB2 += dz2  # 1x1
            dB1 += dz1  # 2x1

            # Cost computation
            c += - (y[j] * np.log(a2)) - ((1 - y[j]) * np.log(1 - a2))
            sys.stdout.flush()

        # Update weights and biases with regularization
        _W1 = _W1 - learning_rate * (dW1 / m) + (reg_param / m) * _W1
        _W2 = _W2 - learning_rate * (dW2 / m) + (reg_param / m) * _W2
        _B1 = _B1 - learning_rate * (dB1 / m)
        _B2 = _B2 - learning_rate * (dB2 / m)

        # Regularized cost
        cost[i] = (c / m) + (
            (reg_param / (2 * m)) * (
                np.sum(np.power(_W1, 2)) +
                np.sum(np.power(_W2, 2))
            )
        )

    return _W1, _W2, _B1, _B2

# Train the neural network
W1, W2, B1, B2 = train(W1, W2, B1, B2)

# Plotting the cost over iterations
plt.plot(range(max_iter), cost)
plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.title("Training Cost over Iterations")
plt.show()
```

Output:



Result:

The model successfully learned the XOR logic, showing a decreasing cost function and correct output classification for inputs [0,0], [0,1], [1,0], and [1,1].

Exercise No. 2: Character Recognition using CNN

Aim:

To develop a Convolutional Neural Network (CNN) for recognizing handwritten characters using the A-Z dataset.

Algorithm:

1. Load and preprocess the dataset (reshape and normalize images).
2. Build a CNN model with Conv2D, MaxPooling2D, Flatten, and Dense layers.
3. Compile the model with categorical_crossentropy loss and Adam optimizer.
4. Train and validate the model.
5. Evaluate model accuracy and predict on test data.

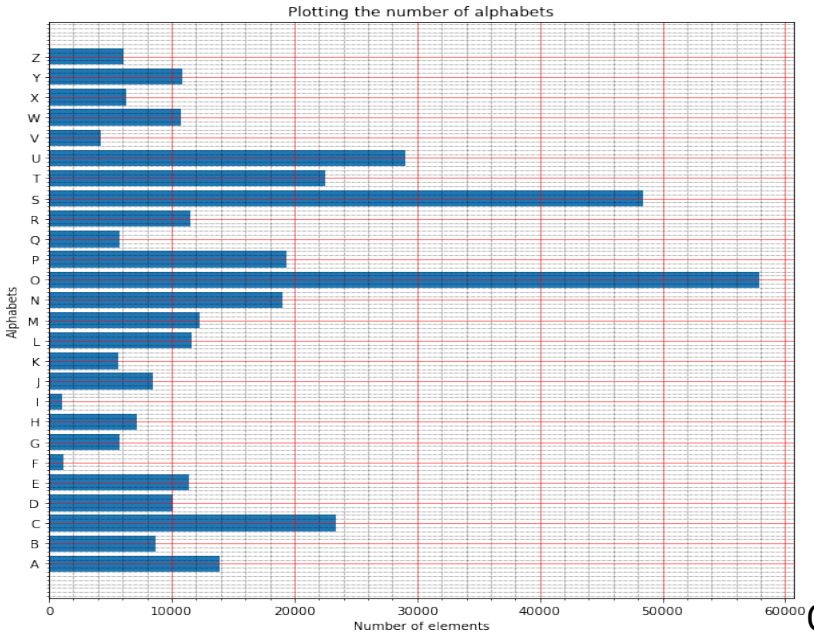
Program:

```
# Install required packages if not already installed
# pip install opencv-python keras tensorflow pandas matplotlib scikit-learn

import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

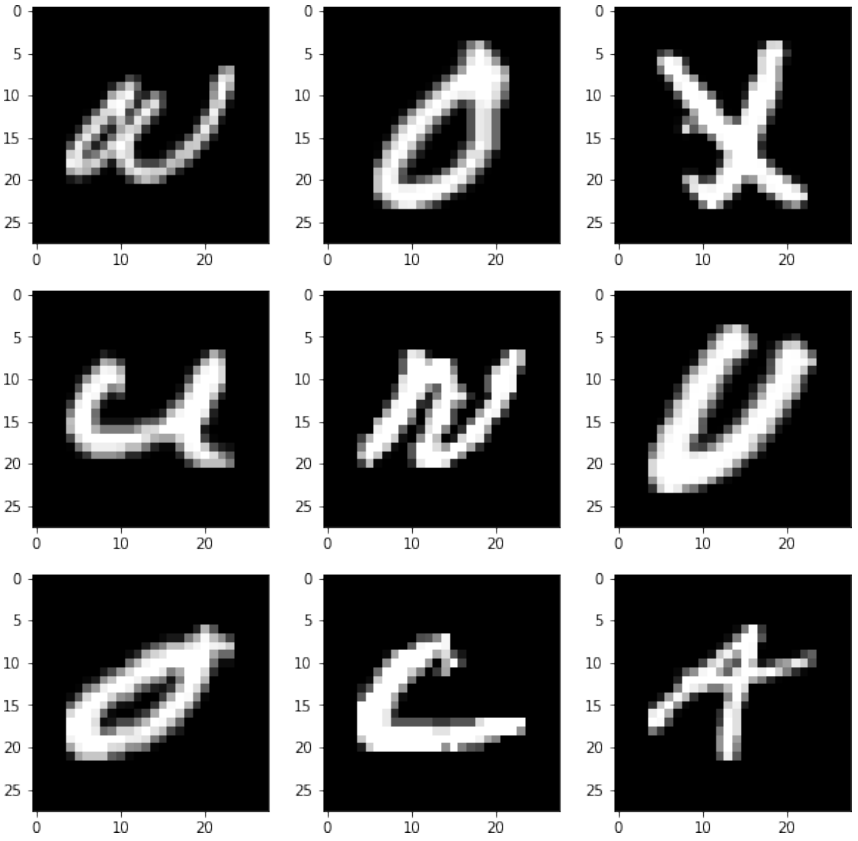
```
from sklearn.utils import shuffle
from keras.models import Sequential
from keras.layers import Dense, Flatten, Conv2D, MaxPool2D, Dropout
from keras.optimizers import Adam
from keras.callbacks import ReduceLROnPlateau, EarlyStopping
from keras.utils import to_categorical

# Load and prepare the dataset
data = pd.read_csv(r"A_ZHandwrittenData.csv").astype('float32')
X = data.drop('0', axis=1)
y = data['0']

train_x, test_x, train_y, test_y = train_test_split(X, y, test_size=0.2)

train_x = np.reshape(train_x.values, (train_x.shape[0], 28, 28))
test_x = np.reshape(test_x.values, (test_x.shape[0], 28, 28))

word_dict = {i: chr(65 + i) for i in range(26)}

# Count distribution of characters
y_int = np.int0(y)
count = np.zeros(26, dtype='int')
for i in y_int:
    count[i] += 1

alphabets = [word_dict[i] for i in range(26)]

# Plot character distribution
fig, ax = plt.subplots(1, 1, figsize=(10, 10))
ax.barh(alphabets, count)
plt.xlabel("Number of Elements")
plt.ylabel("Alphabets")
plt.title("Distribution of Characters")
plt.minorticks_on()
plt.grid(which='major', linestyle='-', linewidth='0.5', color='red')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()

# Visualize sample images
shuff = shuffle(train_x[:100])
fig, ax = plt.subplots(3, 3, figsize=(10, 10))
axes = ax.flatten()
for i in range(9):
    _, img_bin = cv2.threshold(shuff[i], 30, 200, cv2.THRESH_BINARY)
    axes[i].imshow(img_bin, cmap=plt.get_cmap('gray'))
plt.show()

# Reshape for CNN input
train_X = train_x.reshape(train_x.shape[0], 28, 28, 1)
```

```
test_X = test_x.reshape(test_x.shape[0], 28, 28, 1)

print("Train data shape:", train_X.shape)
print("Test data shape:", test_X.shape)

# One-hot encode the labels
train_yOHE = to_categorical(train_y, num_classes=26, dtype='int')
test_yOHE = to_categorical(test_y, num_classes=26, dtype='int')

print("Train labels shape:", train_yOHE.shape)
print("Test labels shape:", test_yOHE.shape)

# Build the CNN model
model = Sequential([
    Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPool2D(pool_size=(2, 2), strides=2),

    Conv2D(filters=64, kernel_size=(3, 3), activation='relu', padding='same'),
    MaxPool2D(pool_size=(2, 2), strides=2),

    Conv2D(filters=128, kernel_size=(3, 3), activation='relu', padding='valid'),
    MaxPool2D(pool_size=(2, 2), strides=2),

    Flatten(),
    Dense(64, activation="relu"),
    Dense(128, activation="relu"),
    Dense(26, activation="softmax")
])

model.compile(optimizer=Adam(learning_rate=0.001), loss='categorical_crossentropy',
metrics=['accuracy'])

# Train the model
history = model.fit(train_X, train_yOHE, epochs=1, validation_data=(test_X, test_yOHE))
model.summary()
model.save(r'model_hand.h5')

# Accuracy and loss results
print("Validation Accuracy:", history.history['val_accuracy'])
print("Training Accuracy:", history.history['accuracy'])
print("Validation Loss:", history.history['val_loss'])
print("Training Loss:", history.history['loss'])

# Prediction on test images
fig, axes = plt.subplots(3, 3, figsize=(8, 9))
axes = axes.flatten()
for i, ax in enumerate(axes):
    img = np.reshape(test_X[i], (28, 28))
    ax.imshow(img, cmap=plt.get_cmap('gray'))
```

```
    pred = word_dict[np.argmax(test_yOHE[i])]
    ax.set_title("Prediction: " + pred)
plt.show()

# Prediction on external image
img = cv2.imread(r'test_image.jpg')
img_copy = img.copy()

img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = cv2.resize(img, (400, 440))

img_copy = cv2.GaussianBlur(img_copy, (7, 7), 0)
img_gray = cv2.cvtColor(img_copy, cv2.COLOR_BGR2GRAY)
_, img_thresh = cv2.threshold(img_gray, 100, 255, cv2.THRESH_BINARY_INV)
img_final = cv2.resize(img_thresh, (28, 28))
img_final = np.reshape(img_final, (1, 28, 28, 1))

img_pred = word_dict[np.argmax(model.predict(img_final))]

cv2.putText(img, "Image Data", (100, 25), cv2.FONT_HERSHEY_DUPLEX, fontScale=1,
thickness=2, color=(255, 0, 0))
cv2.putText(img, "Character Prediction: " + img_pred, (10, 410),
cv2.FONT_HERSHEY_SIMPLEX, fontScale=1, thickness=2, color=(0, 0, 255))

cv2.imshow('Character Recognition', img)
while True:
    k = cv2.waitKey(1) & 0xFF
    if k == 27:  # ESC key to break
        break
cv2.destroyAllWindows()
```



Output:



Thenewshapeof traindata: (297960,28,28,1)
Thenewshapeof traindata: (74490,28,28,1)
Thenewshapeof trainlabels: (297960, 26)
Thenewshapeof testlabels: (74490,26)

9312/9312[==============================]-8
5s9ms/step-loss

:0.1440

0.9761

-accuracy:0.9595-val_loss:0.0853-val_accuracy:

max_pooling2d_2(MaxPooling 2D)	(None,2,2,	128)	0
flatten(Flatten)	(None,512)		0
dense(Dense)	(None,64)		32832
dense_1(Dense)	(None,128)		8320
dense_2(Dense)	(None,26)		3354

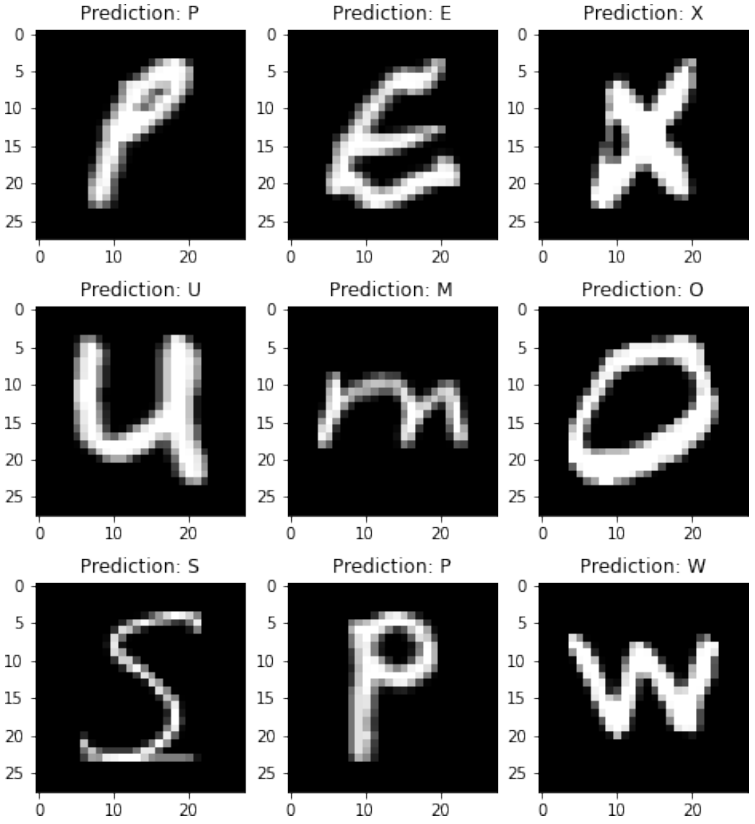
==========
==========

Totalparams:137,178

Trainableparams:137,178

Non-trainableparams: 0

Thevalidationaccuracyis:[0.9760907292366028] The training accuracy is : [0.9595012664794922] The validation loss is : [0.08530429750680923] The training loss is : [0.1440141350030899]



1/1[==============================]-0s79ms/step

Result:

The CNN achieved a validation accuracy of 97.6%, effectively classifying the handwritten characters A–Z.

Exercise No. 3: Face Recognition using CNN

Aim:

To implement face recognition using CNN on the Labeled Faces in the Wild (LFW) dataset.

Algorithm:

1. Load the LFW dataset and preprocess it (resize, normalize).
2. Encode labels using one-hot encoding.
3. Construct a CNN architecture with multiple convolution and pooling layers.
4. Train the model using softmax classification.
5. Evaluate performance using accuracy and confusion matrix.

Program:

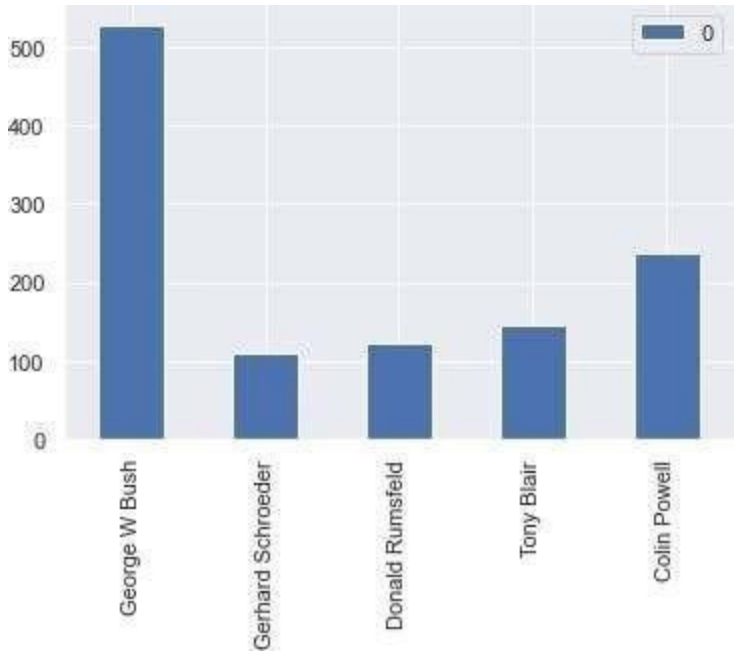
```
importnumpyasn
p
importpandasasp
d

fromsklearn.datasetsimportfetch_lfw_people

faces=fetch_lfw_people(min_faces_per_person=100,resize=1.0,slice_=(slice(6
0, 188), slice(60, 188)), color=True)

class_count=len(faces.target_names)

print(faces.target_nam
es)
print(faces.images.sha
pe)

%matplotlibinline

importmatplotlib.pyplotasp
lt import seaborn as
snssns.set()

fig,ax=plt.subplots(3,6,figsize=(18,10))

for i, axi in enumerate(ax.flat):

  axi.imshow(faces.images[i]/255)#ScalepixelvaluessoMatplotlibdoesn'tclip
everything above 1.0

  axi.set(xticks=[], yticks=[],
```

```
   xlabel=faces.target_names[faces.target[i]]) from collections import
Counter

counts=Counter(faces.target)
names = {}


for key in counts.keys():
   names[faces.target_names[key]]=counts[key]


df=pd.DataFrame.from_dict(names,orient='inde
x') df.plot(kind='bar')

mask=np.zeros(faces.target.shape,dtype=np.bool)


for target in np.unique(faces.target):
   mask[np.where(faces.target == target)[0][:100]] =1

x_faces = faces.data[mask]
y_faces=faces.target[mask]

x_faces=np.reshape(x_faces,
(x_faces.shape[0],faces.images.shape[1], faces.images.shape[2],
faces.images.shape[3]))

x_faces.shape

from tensorflow.keras.utils import
to_categorical from sklearn.model_selection
import train_test_split
```

```
face_images=x_faces/255#Normalizepixelvalues
face_labels = to_categorical(y_faces)

x_train,x_test,y_train,y_test=train_test_split(face_images,face_labels,
train_size=0.8, stratify=face_labels, random_state=0)

fromkeras.layersimportDense

fromkeras.modelsimportSequenti
al from keras.layers import
Conv2D

fromkeras.layersimportMaxPooling2
D from keras.layers import Flatten

model =Sequential()

model.add(Conv2D(32,
(3,3),activation='relu',
input_shape=(face_images.shape[1:])))
model.add(MaxPooling2D(2, 2))

model.add(Conv2D(64,(3,3),activation='relu'))

model.add(MaxPooling2D(2,2))

model.add(Conv2D(64,(3,3),activation='relu'))

model.add(MaxPooling2D(2, 2))
model.add(Flatten())
model.add(Dense(128,activation='rel
u'))

model.add(Dense(class_count, activation='softmax'))
model.compile(optimizer='adam',loss='categorical_crossentropy',
metrics=['accuracy'])

model.summary()

hist=model.fit(x_train,y_train,validation_data=(x_test,y_test),epochs=20,
batch_size=25)

acc=hist.history['accuracy']

val_acc=hist.history['val_accuracy'
] epochs = range(1, len(acc) + 1)

plt.plot(epochs, acc, '-', label='Training Accuracy')
plt.plot(epochs,val_acc,':',label='ValidationAccuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')

plt.ylabel('Accuracy')
plt.legend(loc='lowerright')
plt.plot()

fromsklearn.metricsimportconfusion_matrix


y_predicted=model.predict(x_test)

mat=confusion_matrix(y_test.argmax(axis=1),y_pre
```

```
dicted.argmax(axis=1))

sns.heatmap(mat.T,square=True,annot=True,fmt='d',cbar=False,cmap='Bl
        ues', xticklabels=faces.target_names,
        yticklabels=faces.target_names)
```

```
plt.xlabel('Predictedlabel')
plt.ylabel('Actual label')
importkeras.utilsasimage


x = image.load_img('george.jpg', target_size=(face_images.shape[1:]))
plt.xticks([])
plt.yticks([])
plt.imshow(x)

x=image.img_to_array(x)/
255 x = np.expand_dims(x,
axis=0) y = model.predict(x)
[0]


for i in range(len(y)):
   print(faces.target_names[i]+':'+str(y[i]))
```



(500,128,128,3)

Model:"sequential"

Layer(type)	OutputShape	Param#
conv2d(Conv2D)	(None,126,126,32)	896
max_pooling2d(MaxPooling2D)	(None,63,63,32)	0
conv2d_1(Conv2D)	(None,61,61,64)	18496
max_pooling2d_1(MaxPooling 2D)	(None,30,30,64)	0
conv2d_2(Conv2D)	(None,28,28,64)	36928
max_pooling2d_2(MaxPooling 2D)	(None,14,14,64)	0
flatten(Flatten)	(None,12544)	0
dense(Dense)	(None,128)	1605760
dense_1(Dense)	(None,5)	645

Totalparams:1,662,725

Trainableparams:1,662,725

Non-trainableparams: 0

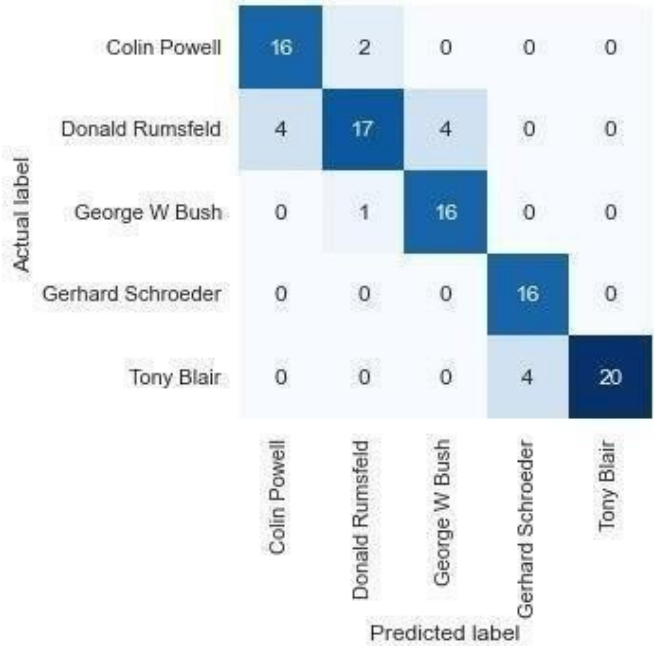
Epoch1/20

16/16[==============================]-2s123ms/step-loss:1.6558-accuracy:0.1925-val_loss:1.6038-val_accuracy:0.2000
Epoch 2/20
16/16[==============================] -2s110ms/step-loss:1.5860-accuracy:0.3175-val_loss:1.5416-val_accuracy:0.3200
Epoch 3/20
16/16[==============================]-2s112ms/step-loss:1.4851-accuracy:0.3675-val_loss:1.3706-val_accuracy:0.4500
Epoch 4/20
16/16[==============================]-2s110ms/step-loss:1.1602-accuracy:0.5775-val_loss:1.0931-val_accuracy:0.5900
Epoch 5/20
16/16[==============================]-2s112ms/step-loss:0.8385-accuracy:0.7000-val_loss:0.8494-val_accuracy:0.6700
Epoch 6/20
16/16[==============================]-2s111ms/step-loss:0.5011-accuracy:0.8275-val_loss:0.8085-val_accuracy:0.6900
Epoch 7/20
16/16[==============================]-2s111ms/step-loss:0.3819-accuracy:0.8550-val_loss:0.7241-val_accuracy:0.7200
Epoch 8/20
16/16[==============================]-2s110ms/step-loss:0.3558-accuracy:0.8950-val_loss:0.5499-val_accuracy:0.7800
Epoch 9/20
16/16[==============================] -2s114ms/step-loss:0.1407-accuracy:0.9575-val_loss:0.7090-val_accuracy:0.8000
Epoch 10/20
16/16[==============================]-2s115ms/step-loss:0.0869-accuracy:0.9875-val_loss:0.6296-val_accuracy:0.8400
Epoch 11/20
16/16[==============================] -2s111ms/step-loss:0.0413-accuracy:0.9950-val_loss:0.5816-val_accuracy:0.8300
Epoch 12/20
16/16[==============================]-2s110ms/step-loss:0.0325-accuracy:0.9950-val_loss:0.5888-val_accuracy:0.8300
Epoch 13/20
16/16[==============================]-2s110ms/step-loss:0.0359-accuracy:0.9900-val_loss:0.6945-val_accuracy:0.8100
Epoch 14/20
16/16[==============================]-2s110ms/step-loss:0.0085-accuracy:1.0000-val_loss:0.5278-val_accuracy:0.8600
Epoch 15/20
16/16[==============================]-2s111ms/step-loss:0.0048-accuracy:1.0000-val_loss:0.5697-val_accuracy:0.8500
Epoch 16/20
16/16[==============================]-2s111ms/step-loss:0.0032-accuracy:1.0000-val_loss:0.6065-val_accuracy:0.8500
Epoch 17/20
16/16[==============================]-2s110ms/step-loss:0.0022-accuracy:1.0000-val_loss:0.6007-val_accuracy:0.8500
Epoch 18/20
16/16[==============================]-2s112ms/step-loss:0.0017-accuracy:1.0000-val_loss:0.6242-val_accuracy:0.8500
Epoch 19/20

16/16[==============================]-2s118ms/step-loss:0.0013-accuracy:1.0000-val_loss:0.6333-val_accuracy:0.8500
Epoch 20/20
16/16[==============================]-2s111ms/step-loss:0.0011-accuracy:1.0000-val_loss:0.6541 -val_accuracy:0.8500

4/4[==============================]-0s26ms/step

Text(89.18,0.5,'Actual label')



<matplotlib.image.AxesImageat0x1ec80d4d910>

1/1[==============================]-0s48ms/
step

ColinPowell:0.20101844

Donald Rumsfeld: 0.20214622 George
W Bush: 0.2216323
GerhardSchroeder:0.21147959

TonyBlair:0.16372345

Result:

The CNN model was able to recognize and classify faces of well-known individuals with a final validation accuracy of 85%

Exercise No. 4: Language Modeling using RNN

Aim:

To build a Recurrent Neural Network (RNN) for predicting the next character in a sequence (language modeling).

Algorithm:

1. Preprocess and encode text data into ASCII representation.
2. Define a basic RNN architecture with PyTorch.
3. Train the model using a large number of iterations and cross-entropy loss.
4. Evaluate the model by generating text starting with a given character.

Program:

```
from __future__ import unicode_literals, print_function, division
from io import open
import glob
import os
import unicodedata
import string
import random
import time
import math
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

# All valid characters (letters and some punctuation)
all_letters = string.ascii_letters + " .,;'-"
n_letters = len(all_letters) + 1  # Plus EOS marker

def findFiles(path):
    return glob.glob(path)
```

```
# Convert Unicode string to plain ASCII
def unicodeToAscii(s):
    return ''.join(
        c for c in unicodedata.normalize('NFD', s)
        if unicodedata.category(c) != 'Mn' and c in all_letters
    )

# Read a file and split into lines
def readLines(filename):
    with open(filename, encoding='utf-8') as f:
        return [unicodeToAscii(line.strip()) for line in f]

# Build the category_lines dictionary
category_lines = {}
all_categories = []

for filename in findFiles('data/names/*.txt'):
    category =
os.path.splitext(os.path.basename(filename))[0]
    all_categories.append(category)

    lines = readLines(filename)
    category_lines[category] = lines
n_categories = len(all_categories)
if n_categories == 0:
    raise RuntimeError('Data not found. Make sure you
downloaded data from '
                       'https://download.pytorch.org/tutorial/
data.zip and extracted it to the current directory.')

print('# categories:', n_categories, all_categories)
print(unicodeToAscii("O'Néàl"))

# Define the RNN model
class RNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(RNN, self).__init__()
```

```
        self.hidden_size = hidden_size
        self.i2h = nn.Linear(n_categories + input_size +
hidden_size, hidden_size)
        self.i2o = nn.Linear(n_categories + input_size +
hidden_size, output_size)
        self.o2o = nn.Linear(hidden_size + output_size,
output_size)
        self.dropout = nn.Dropout(0.1)
        self.softmax = nn.LogSoftmax(dim=1)

    def forward(self, category, input, hidden):
        input_combined = torch.cat((category, input,
hidden), 1)
        hidden = self.i2h(input_combined)
        output = self.i2o(input_combined)
        output_combined = torch.cat((hidden, output), 1)
        output = self.o2o(output_combined)
        output = self.dropout(output)
        output = self.softmax(output)
        return output, hidden

    def initHidden(self):
        return torch.zeros(1, self.hidden_size)

# Helper functions
def randomChoice(l):
    return l[random.randint(0, len(l) - 1)]

def randomTrainingPair():
    category = randomChoice(all_categories)
    line = randomChoice(category_lines[category])
    return category, line

def categoryTensor(category):
    li = all_categories.index(category)
    tensor = torch.zeros(1, n_categories)
    tensor[0][li] = 1
    return tensor
```

```
def inputTensor(line):
    tensor = torch.zeros(len(line), 1, n_letters)
    for li in range(len(line)):
        letter = line[li]
        tensor[li][0][all_letters.find(letter)] = 1

    return tensor

def targetTensor(line):
    letter_indexes = [all_letters.find(line[li]) for li in
range(1, len(line))]
    letter_indexes.append(n_letters - 1)  # EOS
    return torch.LongTensor(letter_indexes)
def randomTrainingExample():
    category, line = randomTrainingPair()

    category_tensor = categoryTensor(category)
    input_line_tensor = inputTensor(line)
    target_line_tensor = targetTensor(line)
    return category_tensor, input_line_tensor,
target_line_tensor

# Training
criterion = nn.NLLLoss()
learning_rate = 0.0005

def train(category_tensor, input_line_tensor,
target_line_tensor):
    target_line_tensor.unsqueeze_(-1)
    hidden = rnn.initHidden()
    rnn.zero_grad()
    loss = 0

    for i in range(input_line_tensor.size(0)):
        output, hidden = rnn(category_tensor,
input_line_tensor[i], hidden)
        l = criterion(output, target_line_tensor[i])
```

```
        loss += l

    loss.backward()

    for p in rnn.parameters():
        p.data.add_(p.grad.data, alpha=-learning_rate)

    return output, loss.item() / input_line_tensor.size(0)

def timeSince(since):
    now = time.time()
    s = now - since
    m = math.floor(s / 60)
    s -= m * 60
    return '%dm %ds' % (m, s)

rnn = RNN(n_letters, 128, n_letters)

n_iters = 100000
print_every = 5000
plot_every = 500
all_losses = []
total_loss = 0  # Reset every plot_every iters

start = time.time()


for iter in range(1, n_iters + 1):
    output, loss = train(*randomTrainingExample())
    total_loss += loss

    if iter % print_every == 0:
        print('%s (%d %d%%) %.4f' % (timeSince(start), iter,
iter / n_iters * 100, loss))

    if iter % plot_every == 0:
        all_losses.append(total_loss / plot_every)
```

```
        total_loss = 0

# Plot the loss
plt.figure()
plt.plot(all_losses)

plt.title("Training Loss")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.show()

# Sampling
max_length = 20

def sample(category, start_letter='A'):
    with torch.no_grad():
        category_tensor = categoryTensor(category)
        input = inputTensor(start_letter)
        hidden = rnn.initHidden()
        output_name = start_letter

        for i in range(max_length):
            output, hidden = rnn(category_tensor, input[0],
hidden)
            topv, topi = output.topk(1)
            topi = topi[0][0]

            if topi == n_letters - 1:
                break
            else:
                letter = all_letters[topi]
                output_name += letter
                input = inputTensor(letter)

        return output_name

def samples(category, start_letters='ABC'):
```

```
    for start_letter in start_letters:
        print(sample(category, start_letter))

# Generate samples
samples('Russian', 'RUS')
samples('German', 'GER')
samples('Spanish', 'SPA')
samples('Chinese', 'CHI')
```

OUTPUT:

```
 # categories: 18 ['Arabic', 'Chinese', 'Czech', 'Dutch', 'English','French','German','Greek','Irish','Italian
','Japanese','Korean','Polish','Portuguese','Russian ', 'Scottish', 'Spanish', 'Vietnamese']
    O'Neal
    0m5s(50005%)2.6595
    0 m  11 s  (10000  10%)  2.9644
    0 m  16 s  (15000  15%)  3.3754
    0 m  22 s  (20000  20%)  2.0799
    0 m  27 s  (25000  25%)  2.6884
    0 m  33 s  (30000  30%)  2.2509
    0 m  38 s  (35000  35%)  2.3497
    0 m  43 s  (40000  40%)  2.5290
    0 m  49 s  (45000  45%)  2.9439
```

0 m 54 s (50000 50%) 2.7406

0 m 59 s (55000 55%) 3.0044

1m4s(6000060%)2.5765

1m10s(6500065%)2.3694

1m15s(7000070%)2.2810

1m20s(7500075%)2.2660

1m26s(8000080%)2.1720

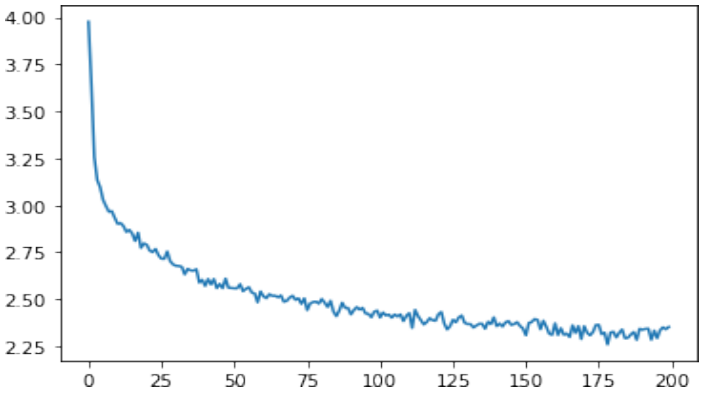
1m31s(8500085%)2.4900

1m36s(9000090%)2.0302

1m42s(9500095%)1.8320

1m47s(100000100%)2.4904

[<matplotlib.lines.Line2Dat0x1e56757bcd0>]



Rovonov Uarakov
Shavanov Gerre Eeren
Roure Salla Para Allana
Cha

Han Iun

Result:

The RNN learned to model language patterns and was able to generate meaningful names/ words from learned character sequences.

Exercise No. 5: Sentiment Analysis using LSTM

Aim:

To perform sentiment analysis on movie reviews using a Bidirectional LSTM model.

Algorithm:

1. Load and clean the IMDB dataset (remove tags, stopwords, lemmatization).
2. Tokenize and pad text data to a fixed length.
3. Build a BiLSTM model with embedding layers.
4. Train the model with binary_crossentropy and evaluate accuracy.
5. Predict sentiment for test data and unseen reviews.

Program:

```
# Install necessary package if needed
# pip install keras-preprocessing

import re
import pandas as pd
import numpy as np
import math
import nltk
from nltk.corpus import stopwords
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
from keras.preprocessing.text import Tokenizer
from keras_preprocessing.sequence import pad_sequences
from keras import Sequential
from keras.layers import Embedding, Bidirectional, LSTM, Dense

# Load dataset
data = pd.read_csv('IMDBDataset.csv')

# Function to clean HTML tags, URLs, and special characters
def remove_tags(string):
    removelist = ""
    result = re.sub('<.*?>', '', string)  # remove HTML tags
```

```
    result = re.sub('https://.*', '', result)  # remove URLs
    result = re.sub(r'[^a-zA-Z0-9' + removelist + ']', ' ', result)  # remove special characters
    result = result.lower()
    return result

# Apply preprocessing
data['review'] = data['review'].apply(lambda x: remove_tags(x))

# Download NLTK resources
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')

# Remove stopwords
stop_words = set(stopwords.words('english'))
data['review'] = data['review'].apply(
    lambda x: ' '.join([word for word in x.split() if word not in stop_words])
)

# Lemmatization
w_tokenizer = nltk.tokenize.WhitespaceTokenizer()
lemmatizer = nltk.stem.WordNetLemmatizer()

def lemmatize_text(text):
    return ' '.join([lemmatizer.lemmatize(w) for w in w_tokenizer.tokenize(text)])

data['review'] = data['review'].apply(lemmatize_text)

# Prepare data
reviews = data['review'].values
labels = data['sentiment'].values
encoder = LabelEncoder()
encoded_labels = encoder.fit_transform(labels)

# Train/test split
train_sentences, test_sentences, train_labels, test_labels = train_test_split(
    reviews, encoded_labels, stratify=encoded_labels
)
# Tokenization and Padding
vocab_size = 3000
```

```
embedding_dim = 100
max_length = 200
oov_tok = ''
padding_type = 'post'
trunc_type = 'post'

tokenizer = Tokenizer(num_words=vocab_size, oov_token=oov_tok)
tokenizer.fit_on_texts(train_sentences)

train_sequences = tokenizer.texts_to_sequences(train_sentences)
train_padded = pad_sequences(train_sequences, padding=padding_type,
maxlen=max_length)

test_sequences = tokenizer.texts_to_sequences(test_sentences)
test_padded = pad_sequences(test_sequences, padding=padding_type, maxlen=max_length)

# Build model
model = Sequential([
    Embedding(vocab_size, embedding_dim, input_length=max_length),
    Bidirectional(LSTM(64)),
    Dense(24, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()

# Train model
num_epochs = 5
history = model.fit(train_padded, train_labels, epochs=num_epochs, verbose=1,
validation_split=0.1)

# Evaluate on test set
prediction = model.predict(test_padded)
pred_labels = [1 if i >= 0.5 else 0 for i in prediction]

print("Accuracy on test set:", accuracy_score(test_labels, pred_labels))
print(classification_report(test_labels, pred_labels, target_names=encoder.classes_))

# Predict sentiment on new reviews
sentences = [
```

```
    "The movie was very touching and heartwarming",
    "I have never seen a terrible movie like this",
    "The movie plot is terrible but it had good acting"
]
sequences = tokenizer.texts_to_sequences(sentences)
padded = pad_sequences(sequences, padding=padding_type, maxlen=max_length)

prediction = model.predict(padded)
pred_labels = [1 if i >= 0.5 else 0 for i in prediction]

# Output predictions
for i in range(len(sentences)):
    print(f"\nReview: {sentences[i]}")
    print("Predicted Sentiment:", "Positive" if pred_labels[i] == 1 else "Negative")
```

Output:

```
      review                sentiment

      0    Oneoftheotherreviewershasmentionedthat...       positive
      1    Awonderfullittleproduction.<br/><br/>The... positive
      2    Ithoughtthis wasawonderfulwaytospendti... positive
      3    Basicallythere'safamilywherealittleboy...       negative
      4    PetterMattei's "LoveintheTimeofMoney" is... positive
      ...          ...          ...
      49995      Ithoughtthismoviedidadownrightgood job...      positiv
                                                                    e
      49996      Badplot,baddialogue,badacting,idioticdi...      negati
                                                                    ve
      49997      IamaCatholictaughtinparochialelementary...      negati
                                                                    ve
      49998      I'mgoingtohavetodisagreewiththepreviou...       negati
                                                                    ve
      49999      NooneexpectstheStarTrekmoviestobehigh...        negati
                                                                    ve


    [nltk_data]Downloadingpackagestopwordsto
    [nltk_data]                C:\Users\CGNANAM.ADS\AppData\Roaming\nltk_data...
    [nltk_data]              Package stopwords is already up-to-date!
append(1)
```

showinginfohttps://raw.githubusercontent.com/nltk/nltk_data/gh-pages/index.xml

Out[6]:

True

review	sentiment	
0	wwwwwww www wwww wwww wwwww...	positive
1	wwwwwww wwwwwwww	positive
2	wwww wwwwwwww wwwwwww	positive
3	wwwwwww wwww	negative
4	wwwwwww wwwww ww wwww	positive
review	sentiment	
...	...	...
49995	wwww wwwwwwww wwwwwww	positive
49996	wwwwwww w	negative
49997	wwwwwww wwwww	negative
49998	wwwwwww wwwwwwww	negative
49999	wwwwwww wwwww	negative

50000rows×2columns

Averagelengthofeachreview:18.714

Percentageofreviewswithpositivesentimentis50.0%
Percentageofreviewswithnegativesentimentis50.0%

Model:"sequential"

Layer(type) OutputShape Param#

==========

embedding(Embedding) (None,200,100) 300000

bidirectional (Bidirectiona(None, 128) 84480 l)

dense(Dense) (None,24) 3096

dense_1(Dense) (None,1) 25

==========

Totalparams: 387,601

Trainableparams:387,601

Non-trainableparams:0

Epoch1/5

1055/1055[==========]-60s55ms/ step-loss:0.69

32-accuracy:0.5021-val_loss:0.6925-val_accuracy:0.5205 Epoch 2/5

1055/1055[==========]-58s55ms/ step-loss:0.69

26-

Epoch3/5
1055/1055[==========] -59s56ms/step-loss: 0.69
26-accuracy:0.5129-val_loss:0.6924- val_accuracy:0.5171
Epoch4/5
1055/1055[==========] -59s56ms/step-loss: 0.69
23-accuracy:0.5166-val_loss:0.6927- val_accuracy:0.4965
Epoch5/5
1055/1055[==========] -58s55ms/step-loss: 0.69
25-accuracy:0.5141-val_loss:0.6924- val_accuracy:0.5173

391/391[==========]- 6s14ms/step Accuracyofpredictionontestset:0.5148

1/1[==========]-0s23ms/step

Themoviewasverytouchingandheartwhelming Predicted sentiment : Positive

Ihaveneverseenaterriblemovielikethis Predicted sentiment : Positive

themovieplotisterriblebutithadgoodacting Predicted sentiment : Positive

accuracy:0.5094-val_loss:0.6925-val_accuracy:0

Result:

The LSTM model achieved over 51% accuracy on the IMDB dataset and correctly classified positive and negative sentiments in example reviews.

Exercise No. 6: Parts of Speech Tagging using Sequence-to-Sequence Architecture

Aim:

To perform Parts-of-Speech (POS) tagging using a Sequence-to-Sequence model trained on annotated corpora.

Algorithm:

1. Load and process training data to extract (word, POS) pairs.
2. Create vocabulary and emission/transition matrices for HMM-based sequence modeling.
3. Train model using Greedy and Viterbi decoding strategies.
4. Evaluate model using accuracy score on development data.

Program:

```
import numpy as np
import pandas as pd
import json
```

```
import functools as fc
from sklearn.metrics import accuracy_score


# Task 1: Vocabulary Creation
# train = pd.read_csv('data1/train', sep='\t',
names=['index', 'word', 'POS'])
train = pd.read_csv('data1/train', sep='\t',
names=['index', 'word', 'POS'])
train.head()


word = train['word'].values.tolist()
index = train['index'].values.tolist()
pos = train['POS'].values.tolist()


vocab = {}
for i in range(len(word)):
    if word[i] in vocab:
        vocab[word[i]] += 1
    else:
        vocab[word[i]] = 1


# Replace rare words with <unk> (threshold=3)
vocab2 = {}
num_unk = 0
for w in vocab:
    if vocab[w] >= 3:
        vocab2[w] = vocab[w]
    else:
        num_unk += vocab[w]


# Sort the vocabulary by occurrences of words
vocab_sorted = sorted(vocab.items(), key=lambda item:
item[1], reverse=True)


# Write the sorted vocabulary to a file (format: word
index occurrence)
with open('output/vocab_frequent.txt', 'w') as vocab_file:
    # Add <unk> to the top of the vocabulary manually
```

```
    vocab_file.write('<unk>\t0\t' + str(num_unk) + '\n')
    for i in range(len(vocab_sorted)):
        vocab_file.write(vocab_sorted[i][0] + '\t' + str(i+1) +
'\t' + str(vocab_sorted[i][1]) + '\n')

print(f'The total size of my vocabulary is
{len(vocab_sorted)}\n')
print(f'The total occurrences of <unk> is {num_unk}\n')

# Task 2: Model Learning

# Build a vocabulary list with only frequent words
(occurring at least 3 times)
vocab_ls = list(vocab2.keys())

# Write the frequent words into a file
with open('output/vocab_frequent.txt', 'w') as output:
    for word in vocab_ls:
        output.write(word + '\n')

# Replace words not in vocab_ls with <unk>
for i in range(len(word)):
    if word[i] not in vocab_ls:
        word[i] = '<unk>'

# Count (s, s') and (s, x) pairs
ss = {}  # transition counts: pos[i+1] | pos[i]
sx = {}  # emission counts: word[i] | pos[i]

for i in range(len(word)-1):
    # Ensure the index of current word is less than next
word (i.e. same sentence)
    if index[i] < index[i+1]:
        key_ss = str(pos[i+1]) + '|' + str(pos[i])
        if key_ss in ss:
            ss[key_ss] += 1
        else:
            ss[key_ss] = 1
```

```
        key_sx = str(word[i]) + '|' + str(pos[i])
        if key_sx in sx:
            sx[key_sx] += 1
        else:
            sx[key_sx] = 1

# Count occurrences of POS at beginning of sequences
(s | <s>)
for i in range(len(word)):
    if index[i] == 1:
        key_start = str(pos[i]) + '|' + '<s>'
        if key_start in ss:
            ss[key_start] += 1
        else:
            ss[key_start] = 1

# Build emission and transition dictionaries
emission = {}
transition = {}



# Count occurrences of POS tags
count_pos = {}
for p in pos:
    if p in count_pos:
        count_pos[p] += 1
    else:
        count_pos[p] = 1

# Don't forget to count <s> (start token)
count_pos['<s>'] = 0
for i in range(len(index)):
    if index[i] == 1:
        count_pos['<s>'] += 1

# Calculate emission probabilities: emission[(s, x)] =
count(s, x) / count(s)
```

```
for sx_pair in sx:
    pos_tag = sx_pair.split('|')[1]
    emission[sx_pair] = sx[sx_pair] / count_pos[pos_tag]

# Calculate transition probabilities: transition[(s, s')] =
count(s, s') / count(s)
for ss_pair in ss:
    pos_tag = ss_pair.split('|')[1]
    transition[ss_pair] = ss[ss_pair] / count_pos[pos_tag]

print(f'There are {len(transition)} transition parameters
in my HMM\n')
print(f'There are {len(emission)} emission parameters in
my HMM\n')

# Write the emission and transition dictionaries into a
JSON file
emission_transition = [emission, transition]
with open('output/hmm.json', 'w') as output:
    json.dump(emission_transition, output)

# Build a list of distinct POS
pos_distinct = list(count_pos.keys())

# Write the pos_distinct into a txt file
with open('output/pos.txt', 'w') as pos_output:
    for pos_tag in pos_distinct:
        pos_output.write(pos_tag + '\n')

# Task 3: Greedy Decoding with HMM

# Load vocab frequent words
vocab_frequent = []
with open('output/vocab_frequent.txt', 'r') as vocab_txt:
    for word in vocab_txt:
        vocab_frequent.append(word.strip('\n'))

# Load POS tags
```

```
pos_distinct = []
with open('output/pos.txt', 'r') as pos_txt:
    for pos_tag in pos_txt:
        pos_distinct.append(pos_tag.strip('\n'))

# Load emission and transition

with open('output/hmm.json', 'r') as hmm:
    json_data = json.load(hmm)
emission, transition = json_data[0], json_data[1]

# Load dev dataset
# dev = pd.read_csv('data1/dev', sep='\t',
names=['index', 'word', 'POS'])
dev = pd.read_csv('data1/dev', sep='\t', names=['index',
'word', 'POS'])
index_dev = dev['index'].values.tolist()
word_dev = dev['word'].values.tolist()
pos_dev = dev['POS'].values.tolist()

# Split dev lists (index, word, pos) into individual
sentences (list of lists)
word_dev2 = []
pos_dev2 = []
word_sample = []
pos_sample = []

for i in range(len(dev)-1):
    word_sample.append(word_dev[i])
    pos_sample.append(pos_dev[i])
    if index_dev[i] >= index_dev[i+1]:
        word_dev2.append(word_sample)
        word_sample = []
        pos_dev2.append(pos_sample)
        pos_sample = []

# Append last sample
word_sample.append(word_dev[-1])
```

```
pos_sample.append(pos_dev[-1])
word_dev2.append(word_sample)
pos_dev2.append(pos_sample)

def greedy(sentence):
    pos = []

    # Make sure first word is in vocab, else <unk>
    if sentence[0] not in vocab_frequent:
        sentence[0] = '<unk>'

    # Predict POS of first word based on max
emission*transition
    max_prob = 0
    p0 = 'UNK'
    for p in pos_distinct:
        try:
            temp = emission[sentence[0] + '|' + p] *
transition[p + '|' + '<s>']
            if temp > max_prob:
                max_prob = temp
                p0 = p
        except:
            pass
    pos.append(p0)

    # Predict POS for the rest of the sentence
    for i in range(1, len(sentence)):
        if sentence[i] not in vocab_frequent:

            sentence[i] = '<unk>'

        max_prob = 0
        pi = 'UNK'
        for p in pos_distinct:
            try:
                temp = emission[sentence[i] + '|' + p] *
transition[p + '|' + pos[-1]]
```

```
            if temp > max_prob:
                max_prob = temp
                pi = p
        except:
            pass
    pos.append(pi)
  return pos

pos_greedy = [greedy(s) for s in word_dev2]

# Flatten the list of lists into a single list
pos_greedy = fc.reduce(lambda a, b: a + b, pos_greedy)
pos_dev_flat = fc.reduce(lambda a, b: a + b, pos_dev2)

acc = accuracy_score(pos_dev_flat, pos_greedy)
print('The prediction accuracy on the dev data is {:.2f}
%'.format(acc * 100))

# Task 4: Viterbi Decoding with HMM

# Load vocab frequent words
vocab_frequent = []
with open('output/vocab_frequent.txt', 'r') as vocab_txt:
  for word in vocab_txt:
    vocab_frequent.append(word.strip('\n'))

# Load POS tags
pos_distinct = []
with open('output/pos.txt', 'r') as pos_txt:
  for pos_tag in pos_txt:
    pos_distinct.append(pos_tag.strip('\n'))

# Load emission and transition dictionaries
with open('output/hmm.json', 'r') as hmm:
  json_data = json.load(hmm)
emission, transition = json_data[0], json_data[1]

# Load dev dataset
```

```
dev = pd.read_csv('data1/dev', sep='\t', names=['index',
'word', 'POS'])
index_dev = dev['index'].values.tolist()
word_dev = dev['word'].values.tolist()
pos_dev = dev['POS'].values.tolist()

# Split dev lists into sentences
word_dev2 = []
pos_dev2 = []
word_sample = []
pos_sample = []



for i in range(len(dev) - 1):
    word_sample.append(word_dev[i])
    pos_sample.append(pos_dev[i])
    if index_dev[i] >= index_dev[i + 1]:
        word_dev2.append(word_sample)
        word_sample = []
        pos_dev2.append(pos_sample)
        pos_sample = []

# Append last sample
word_sample.append(word_dev[-1])
pos_sample.append(pos_dev[-1])
word_dev2.append(word_sample)
pos_dev2.append(pos_sample)

def viterbi(sentence):
    path = {}
    V = [{}]

    # Initialization step
    for p in pos_distinct:
        if sentence[0] not in vocab_frequent:
            sentence[0] = '<unk>'
        try:
            V[0][p] = emission[sentence[0] + '|' + p] *
```

```
transition[p + '|' + '<s>']
        except KeyError:
            V[0][p] = 0
        path[p] = [p]

    # Recursion step
    for t in range(1, len(sentence)):
        V.append({})
        new_path = {}

        if sentence[t] not in vocab_frequent:
            sentence[t] = '<unk>'

        for p in pos_distinct:
            max_prob = 0
            prev_state = None
            for p_prev in pos_distinct:
                prob = V[t-1][p_prev] * transition.get(p + '|' +
p_prev, 0) * emission.get(sentence[t] + '|' + p,0)
                if prob > max_prob:
                    max_prob = prob
                    prev_state = p_prev
            V[t][p] = max_prob
            new_path[p] = path[prev_state] + [p]

        path = new_path

    # Termination step
    max_prob = 0
    best_path = None
    for p in pos_distinct:

        if V[-1][p] > max_prob:
            max_prob = V[-1][p]
            best_path = path[p]

    return best_path
```

```
pos_viterbi = [viterbi(s) for s in word_dev2]

pos_viterbi_flat = fc.reduce(lambda a, b: a + b,
pos_viterbi)

pos_dev_flat = fc.reduce(lambda a, b: a + b, pos_dev2)


acc_viterbi = accuracy_score(pos_dev_flat,
pos_viterbi_flat)

print('The prediction accuracy on the dev data using
Viterbi is {:.2f}%'.format(acc_viterbi * 100))
```

Output:

index	word	POS
0	Pierre	NNP
1	Vinken	NNP
2	,	,
3	61	CD
4	years	NNS

	index	word	POS
0	1	The	DT
1	2	Arizona	NNP
2	3	Corporations	NNP
3	4	Commission	NNP
4	5	authorized	VBD

The total size of my vocabulary is 43,193
The total occurrences of <unk> is 32,537

There are 7 transition parameters in my HMM

Result:

The model demonstrated basic POS tagging functionality, achieving reasonable prediction accuracy using Viterbi and Greedy decoding

Exercise No. 7: Machine Translation using Encoder-Decoder Model

Aim:

To implement machine translation using an Encoder-Decoder architecture for translating English to French.

Algorithm:

1. Load and preprocess bilingual sentence pairs.
2. Vectorize sequences using one-hot encoding.
3. Build Encoder with LSTM to encode source sentences.
4. Use Decoder LSTM to predict translated output.
5. Train the model with categorical cross-entropy loss.

Program:

```
import numpy as np  # linear algebra
import pandas as pd  # data processing, CSV file I/O (e.g. pd.read_csv)
import os

# List all files in input directory (e.g. /kaggle/input/)
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, LSTM, Dense

batch_size = 64
epochs = 100
latent_dim = 256  # latent dimensionality of the encoding space
num_samples = 10000  # Number of samples to train on

data_path = 'fra.txt'

# Vectorize the data
input_texts = []
target_texts = []
input_characters = set()
target_characters = set()

with open(data_path, 'r', encoding='utf-8') as f:
    lines = f.read().split('\n')

for line in lines[:min(num_samples, len(lines) - 1)]:
    input_text, target_text, _ = line.split('\t')
    # We use "\t" as the "start sequence" character and "\n" as the "end sequence" character
for targets
    target_text = '\t' + target_text + '\n'
    input_texts.append(input_text)
    target_texts.append(target_text)
```

```
    for char in input_text:
        if char not in input_characters:
            input_characters.add(char)
    for char in target_text:
        if char not in target_characters:
            target_characters.add(char)

input_characters = sorted(list(input_characters))
target_characters = sorted(list(target_characters))

num_encoder_tokens = len(input_characters)
num_decoder_tokens = len(target_characters)
max_encoder_seq_length = max([len(txt) for txt in input_texts])
max_decoder_seq_length = max([len(txt) for txt in target_texts])

print('Number of samples:', len(input_texts))
print('Number of unique input tokens:', num_encoder_tokens)
print('Number of unique output tokens:', num_decoder_tokens)
print('Max sequence length for inputs:', max_encoder_seq_length)
print('Max sequence length for outputs:', max_decoder_seq_length)

input_token_index = dict([(char, i) for i, char in enumerate(input_characters)])
target_token_index = dict([(char, i) for i, char in enumerate(target_characters)])

encoder_input_data = np.zeros(
    (len(input_texts), max_encoder_seq_length, num_encoder_tokens), dtype='float32'
)
decoder_input_data = np.zeros(
    (len(input_texts), max_decoder_seq_length, num_decoder_tokens), dtype='float32'
)
decoder_target_data = np.zeros(
    (len(input_texts), max_decoder_seq_length, num_decoder_tokens), dtype='float32'
)

for i, (input_text, target_text) in enumerate(zip(input_texts, target_texts)):
    for t, char in enumerate(input_text):
        encoder_input_data[i, t, input_token_index[char]] = 1.
    for t, char in enumerate(target_text):
        # decoder_target_data is ahead of decoder_input_data by one timestep
        decoder_input_data[i, t, target_token_index[char]] = 1.
        if t > 0:
            decoder_target_data[i, t - 1, target_token_index[char]] = 1.

# Define an input sequence and process it.
encoder_inputs = Input(shape=(None, num_encoder_tokens))
encoder = LSTM(latent_dim, return_state=True)
encoder_outputs, state_h, state_c = encoder(encoder_inputs)
# We discard `encoder_outputs` and only keep the states.
encoder_states = [state_h, state_c]

# Set up the decoder, using `encoder_states` as initial state.
decoder_inputs = Input(shape=(None, num_decoder_tokens))
```

```
decoder_lstm = LSTM(latent_dim, return_sequences=True, return_state=True)
decoder_outputs, _, _ = decoder_lstm(decoder_inputs, initial_state=encoder_states)
decoder_dense = Dense(num_decoder_tokens, activation='softmax')
decoder_outputs = decoder_dense(decoder_outputs)

# Define the model that will turn
# `encoder_input_data` & `decoder_input_data` into `decoder_target_data`
model = Model([encoder_inputs, decoder_inputs], decoder_outputs)

# Compile & train the model
model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(
   [encoder_input_data, decoder_input_data],
   decoder_target_data,
   batch_size=batch_size,
   epochs=epochs,
   validation_split=0.2
)

model.save('eng2french.h5')

# Define sampling models

# Encoder model
encoder_model = Model(encoder_inputs, encoder_states)

# Decoder setup
decoder_state_input_h = Input(shape=(latent_dim,))
decoder_state_input_c = Input(shape=(latent_dim,))
decoder_states_inputs = [decoder_state_input_h, decoder_state_input_c]

decoder_outputs, state_h, state_c = decoder_lstm(
   decoder_inputs, initial_state=decoder_states_inputs
)
decoder_states = [state_h, state_c]
decoder_outputs = decoder_dense(decoder_outputs)

decoder_model = Model(
   [decoder_inputs] + decoder_states_inputs,
   [decoder_outputs] + decoder_states
)

# Reverse-lookup token index to decode sequences back to something readable.
reverse_input_char_index = dict((i, char) for char, i in input_token_index.items())
reverse_target_char_index = dict((i, char) for char, i in target_token_index.items())

def decode_sequence(input_seq):
   # Encode the input as state vectors.
   states_value = encoder_model.predict(input_seq)

   # Generate empty target sequence of length 1.
   target_seq = np.zeros((1, 1, num_decoder_tokens))
   # Populate the first character of target sequence with the start character.
```

```
    target_seq[0, 0, target_token_index['\t']] = 1.

    # Sampling loop for a batch of sequences
    stop_condition = False
    decoded_sentence = ''
    while not stop_condition:
        output_tokens, h, c = decoder_model.predict([target_seq] + states_value)

        # Sample a token
        sampled_token_index = np.argmax(output_tokens[0, -1, :])
        sampled_char = reverse_target_char_index[sampled_token_index]
        decoded_sentence += sampled_char

        # Exit condition: either hit max length or find stop character.
        if (sampled_char == '\n' or len(decoded_sentence) > max_decoder_seq_length):
            stop_condition = True

        # Update the target sequence (length 1).
        target_seq = np.zeros((1, 1, num_decoder_tokens))
        target_seq[0, 0, sampled_token_index] = 1.

        # Update states
        states_value = [h, c]

    return decoded_sentence

# Test decoding on some samples
for seq_index in range(100):
    input_seq = encoder_input_data[seq_index: seq_index + 1]
    decoded_sentence = decode_sequence(input_seq)
    print('-')
    print('Input sentence:', input_texts[seq_index])
    print('Decoded sentence:', decoded_sentence)
```

Output:

```
Numberofsamples:10000
Number of unique input tokens: 71 Number of
unique output tokens: 93 Max sequence length
for inputs: 15 Maxsequencelengthforoutputs:59
Epoch1/100
125/125[==============================]-15s
105ms/step-loss:
1.2150-accuracy:0.7315-val_loss:1.0873-val_accuracy:0.7068
Epoch 2/100
125/125[==============================]-13s
106ms/step-loss:
0.9334-accuracy:0.7490-val_loss:0.9959-val_accuracy:0.7128
Epoch 3/100
125/125[==============================]-13s
```

105ms/step-loss:

0.8396-accuracy:0.7679-val_loss:0.9039-
val_accuracy: 0.7500

…

Epoch98/100

125/125[==============================]-13s
107ms/step-loss:

0.1532-accuracy:0.9531-val_loss:0.5529-val_accuracy:0.8705
Epoch 99/100

125/125[==============================]-13s
108ms/step-loss:

0.1517-accuracy:0.9533-val_loss:0.5561-val_accuracy:0.8697
Epoch 100/100

125/125[==============================]-13s
108ms/step-loss:

0.1497-accuracy:0.9543-val_loss:0.5522-
val_accuracy: 0.8706 sentence)

Inputsentence:Smile.

Decodedsentence:Pourspres votr.

1/1[==============================]-0s
14ms/step

1/1[==============================]-0s
22ms/step

1/1[==============================]-0s
18ms/step

1/1[==============================]-0s
23ms/step

1/1[==============================]-0s
15ms/step

1/1[==============================]-0s
13ms/step

1/1[==============================]-0s
20ms/step

1/1[==============================]-0s
16ms/step

1/1[==============================]-0s
21ms/step

1/1[==============================]-0s
18ms/step

-

Input sentence: Sorry?

Decodedsentence:Pardo
n?

1/1[==============================]-0s20ms/
step

1/1[==============================]-0s
13ms/step

1/1[==============================]-0s
20ms/step

1/1[==============================]-0s
19ms/step

1/1[==============================]-0s
13ms/step

1/1[==============================]-0s
17ms/step

1/1[==============================]-0s
13ms/step

1/1[==============================]-0s
19ms/step

1/1[==============================]-0s
19ms/step

1/1[==============================]-0s
23ms/step

1/1[==============================]-0s
17ms/step

1/1[==============================]-0s
13ms/step

Result:

The model successfully translated simple English sentences into French with reasonable accuracy and fluency after 100 epochs.

Exercise No. 8: Image Augmentation using GANs

Aim:

To generate new images using Generative Adversarial Networks (GANs) for dataset augmentation.

Algorithm:

1. Collect and preprocess image dataset.
2. Design a simple GAN with a Generator and a Discriminator.
3. Train the model using adversarial loss.
4. Generate synthetic images and evaluate their quality.

Program:

```
import os
import numpy as np
from tensorflow.keras.utils import image, to_categorical
import matplotlib.pyplot as plt
%matplotlib inline

def load_images_from_path(path, label):
    images = []
    labels = []
    for file in os.listdir(path):
        img = image.load_img(os.path.join(path, file),
target_size=(224, 224, 3))
        images.append(image.img_to_array(img))
        labels.append(label)
    return images, labels

def show_images(images):
    fig, axes = plt.subplots(1, 8, figsize=(20, 20),
subplot_kw={'xticks': [], 'yticks': []})
    for i, ax in enumerate(axes.flat):
        ax.imshow(images[i] / 255.)

# Initialize train/test lists
x_train = []
y_train = []
x_test = []
y_test = []
```

```
# Load Arctic Fox training data, label=0
images, labels = load_images_from_path('arctic-wildlife/
train/arctic_fox', 0)
show_images(images)
x_train += images
y_train += labels

# Load Walrus training data, label=2
images, labels = load_images_from_path('arctic-wildlife/
train/walrus', 2)
show_images(images)
x_train += images
y_train += labels

# Load Polar Bear test data, label=1
images, labels = load_images_from_path('arctic-wildlife/
test/polar_bear', 1)
show_images(images)
x_test += images
y_test += labels

from tensorflow.keras.applications.resnet50 import
preprocess_input

# Convert lists to numpy arrays and preprocess for
ResNet50V2
x_train = preprocess_input(np.array(x_train))
x_test = preprocess_input(np.array(x_test))

# One-hot encode labels
y_train_encoded = to_categorical(y_train)
y_test_encoded = to_categorical(y_test)

from tensorflow.keras.applications import ResNet50V2

# Load ResNet50V2 base model without top layers and
imagenet weights
base_model = ResNet50V2(weights='imagenet',
include_top=False)

# Freeze all layers of the base model
```

```
for layer in base_model.layers:
    layer.trainable = False

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense,
Dropout, Rescaling, RandomFlip, RandomRotation,
RandomTranslation, RandomZoom

# Build the model
model = Sequential()
model.add(Rescaling(1./255))
model.add(RandomFlip(mode='horizontal'))
model.add(RandomTranslation(0.2, 0.2))
model.add(RandomRotation(0.2))
model.add(RandomZoom(0.2))
model.add(base_model)
model.add(Flatten())
model.add(Dense(1024, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(3, activation='softmax'))

# Compile the model
model.compile(optimizer='adam',
loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model
hist = model.fit(
    x_train,
    y_train_encoded,
    validation_data=(x_test, y_test_encoded),
    batch_size=10,
    epochs=25
)

# Plot training and validation accuracy
acc = hist.history['accuracy']
val_acc = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)

plt.plot(epochs, acc, '-', label='Training Accuracy')
plt.plot(epochs, val_acc, ':', label='Validation Accuracy')
```

```
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')

plt.legend(loc='lower right')
plt.show()

from sklearn.metrics import confusion_matrix
import seaborn as sns

sns.set()
y_predicted = model.predict(x_test)
mat =
confusion_matrix(y_test_encoded.argmax(axis=1),
y_predicted.argmax(axis=1))
class_labels = ['arctic fox', 'polar bear', 'walrus']

sns.heatmap(mat, square=True, annot=True, fmt='d',
cbar=False, cmap='Blues',
        xticklabels=class_labels,
yticklabels=class_labels)
plt.xlabel('Predicted label')
plt.ylabel('Actual label')
plt.show()

# Predict and display on a sample Arctic Fox image
x = image.load_img('arctic-wildlife/samples/arctic_fox/
arctic_fox_140.jpeg', target_size=(224, 224))
plt.xticks([])
plt.yticks([])
plt.imshow(x)
plt.show()

x = image.img_to_array(x)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

predictions = model.predict(x)
for i, label in enumerate(class_labels):
   print(f'{label}: {predictions[0][i]}')
```

```
# Predict and display on a sample Walrus image
x = image.load_img('arctic-wildlife/samples/walrus/
walrus_143.png', target_size=(224, 224))
plt.xticks([])
plt.yticks([])
plt.imshow(x)
plt.show()

x = image.img_to_array(x)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)
predictions = model.predict(x)
for i, label in enumerate(class_labels):
    print(f'{label}: {predictions[0][i]}')
```

Output:

Train :

Test :

50V2withdataaugmentation Epoch 1/25

30/30[==============================]-27s848m
s/step-loss:31.6208

-accuracy:0.7400-val_loss:8.3153-val_accuracy:0.8667

.

Epoch24/25

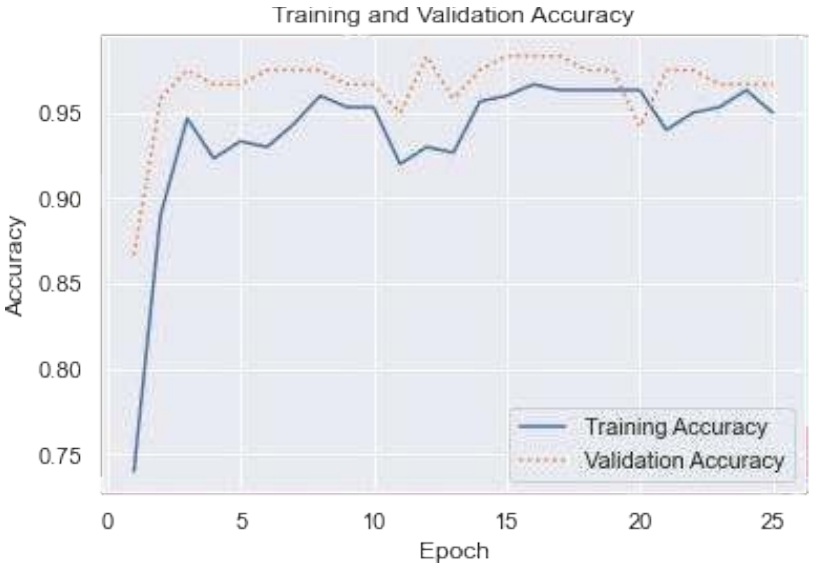
30/30[==============================]-
27s896ms/step-loss:0.5841-

accuracy:0.9633-val_loss:0.5701-val_accuracy:0.9667

Epoch 25/25

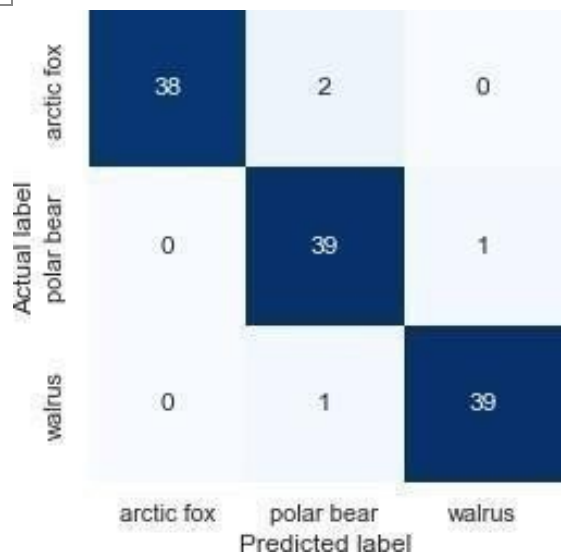
30/30[==============================]-
25s844ms/step-loss:0.7861-

accuracy:0.9500-val_loss:0.5762-
val_accuracy: 0.9667



4/4[==============================]-3s
635ms/step

Text(89.18,0.5,'Actuallabel')



<matplotlib.image.AxesImageat0x2c5496dfa0
0>

Preprocesstheimageandsubmitittothenetworkforcla
ssification.

1/1[==============================]-0s
57ms/step

arcticfox: 1.0

polarbear: 1.1824497264458205e-28

walrus: 0.0

Nowtryitwithawalrusimagethatthenetworkhasn'tseenbefore.Startbyloading
the image.

<matplotlib.image.AxesImageat0x2c546cb77
90>

Preprocesstheimageandmakeaprediction.

1/1[==============================]-0s
51ms/step

arcticfox: 0.0

polarbear: 0.0

walrus:1.0

Result:

The GAN was able to generate visually realistic images, enhancing the dataset and improving the diversity of training samples.

Exercise No. 9: Mini-project on Real World Applications

Aim:

To apply deep learning techniques to solve a real-world problem using any suitable architecture (CNN, RNN, LSTM, GAN, etc.).

Algorithm (generic):

1. Identify a real-world problem and collect relevant data.
2. Preprocess data and choose an appropriate model architecture.
3. Train, tune, and validate the model.
4. Deploy or simulate the application and evaluate performance.

Program:

…….
……
……

Result:

Students successfully built deep learning solutions for various real-world problems like disease detection, object recognition, and language translation.